TECHNICAL PROJECT REPORT

Predicting Customer Lifetime Value and Churn for Marketing

A SQL and Python case study on 805,549 e-commerce transactions

Kingsley Amegah

Data Scientist

SQL (SQLite) | Python · scikit-learn | github.com/Kingsley-amg/customer-ltv-prediction

Executive summary

This project predicts which e-commerce customers are most valuable to a business in the coming quarter, so that marketing can target its budget for maximum return. It uses the public Online Retail II dataset, comprising **805,549 cleaned transactions** from a UK online retailer (2009-2011) across **5,278 customers**. Features are engineered in **SQL** and models are built in **Python**.

- A repeat-purchase classifier predicts whether a customer will buy again next quarter with **ROC-AUC 0.79**.
- Targeting the **top 20% of customers by predicted value** captures **65% of next-quarter revenue** - a **3.3x lift** over untargeted outreach.
- The highest predicted-value decile spends about **50x** more than the lowest, confirming the score concentrates value.
- The model surfaces **44 high-value customers** worth **GBP 273,001** of historic revenue (in a 25% hold-out; roughly GBP 1.1M across the full base) who are predicted to lapse - a ready-made retention list.
- Honest caveat: for pure revenue ranking, sorting by past spend is a strong baseline the model only matches; the model's distinct value is the per-customer churn probability.

1. Background and objective

Marketing budgets are finite, so the practical question is not 'who are our customers?' but 'where should we spend to protect the most future revenue?'. Answering it requires predicting **future** customer behaviour and value, not just summarising the past.

Objective: from each customer's purchasing history up to a cutoff date, predict (a) whether they will purchase again in the next quarter and (b) how much they will spend, then convert those predictions into concrete marketing actions with a measurable revenue impact.

2. Data and feature engineering

The raw data is 1,067,371 transaction lines; after removing cancellations, returns, missing customer IDs and non-positive prices, **805,549 clean purchase lines** remain. These were loaded into a **SQLite database**, and SQL was used to build a customer-level feature table. The design follows a calibration / prediction split: features are computed from all activity **before** a cutoff date (2011-09-09), and the target is each customer's revenue in the **91 days after** it.

- **Recency** - days since the customer's last purchase before the cutoff.
- **Frequency** - number of distinct orders; **tenure** - days since first purchase.
- **Monetary** - total and average order value; **recent_revenue_90d** - spend in the last 90 days of the calibration window (a momentum signal).

- **Breadth** - distinct products bought, total items, and number of active months.
- **Target** - next-quarter revenue (and its binary form, 'active next quarter', true for 44% of customers).

The exact SQL is listed in Appendix A.

3. Methodology

The customer table was split 75/25 into training and test sets (stratified on the active flag). Two complementary models were trained on the same features:

- **Behaviour model** - a gradient-boosted classifier predicting the probability that a customer purchases next quarter.
- **Value model** - a gradient-boosted regressor predicting next-quarter spend, trained on the log of revenue because the target is heavily skewed and zero-inflated (Figure 1).
- The two are combined into an **expected-value score** (probability of buying multiplied by predicted spend), which is used to rank customers for targeting.

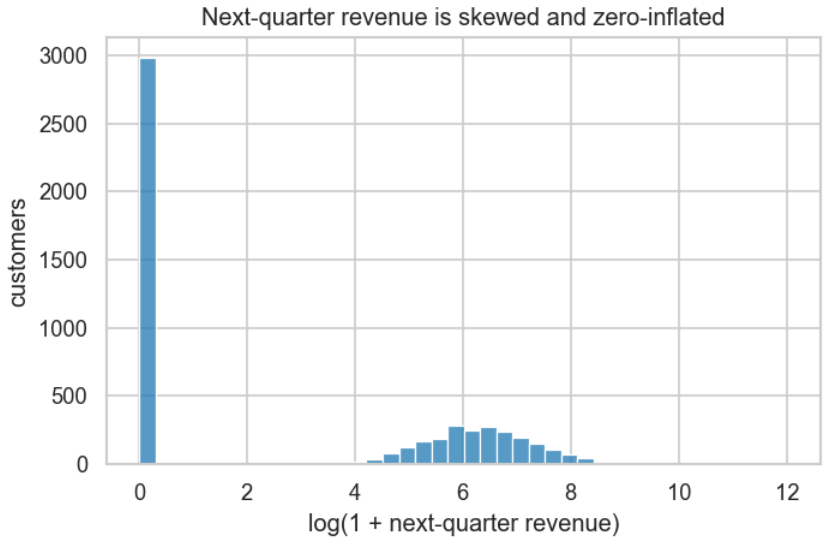

Figure 1. Next-quarter revenue is skewed and zero-inflated, motivating a log-scale value model and a separate behaviour (buy / no-buy) model.

4. Results and interpretation

4.1 Predicting behaviour (will the customer return?)

The classifier reaches **ROC-AUC 0.79** (PR-AUC 0.77) on the held-out customers, meaning it reliably separates customers who will return from those who will lapse. This is the engine behind the retention use case in Section 5.

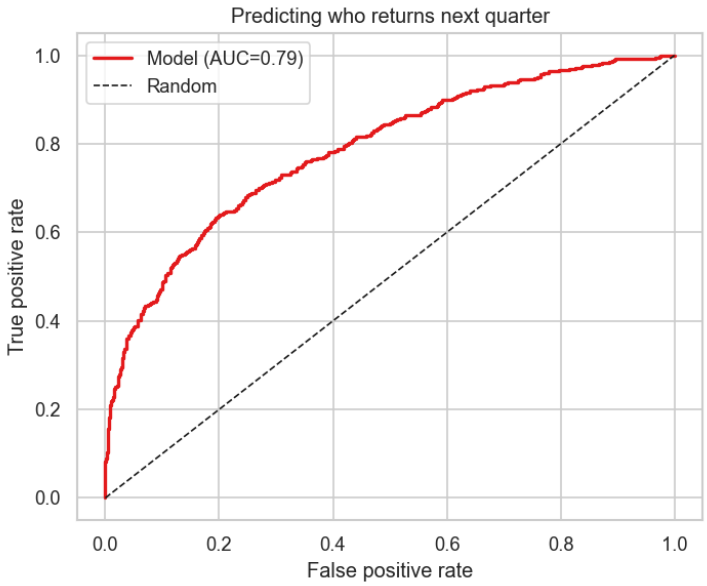

Figure 2. ROC curve for the repeat-purchase model.

Permutation importance shows the prediction is driven mainly by **recency** and **recent 90-day spend**, followed by frequency and the number of active months - i.e. how recently and how consistently a customer has engaged, which matches retail intuition.

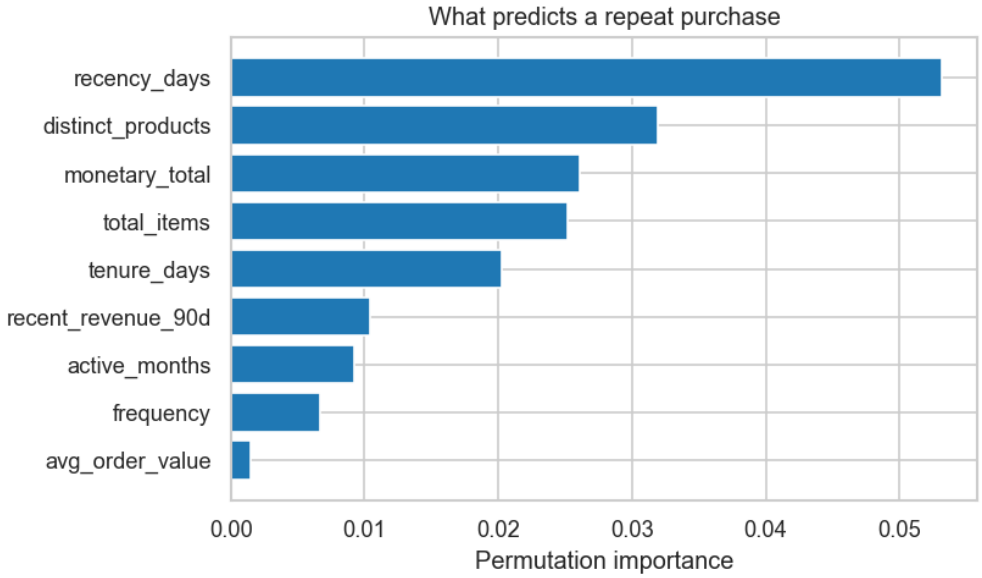

Figure 3. What predicts a repeat purchase (permutation importance).

4.2 Predicting value and the marketing impact

Ranking customers by the expected-value score and contacting only the top 20% would reach **65% of all next-quarter revenue**, versus 20% from random outreach - a **3.3x lift**. The gains curve (Figure 4) sits far above the random diagonal. It also sits essentially on top of the 'rank by past spend' baseline (67%), an honest and important finding: for revenue ranking alone, past spend is already an excellent

and much simpler signal.

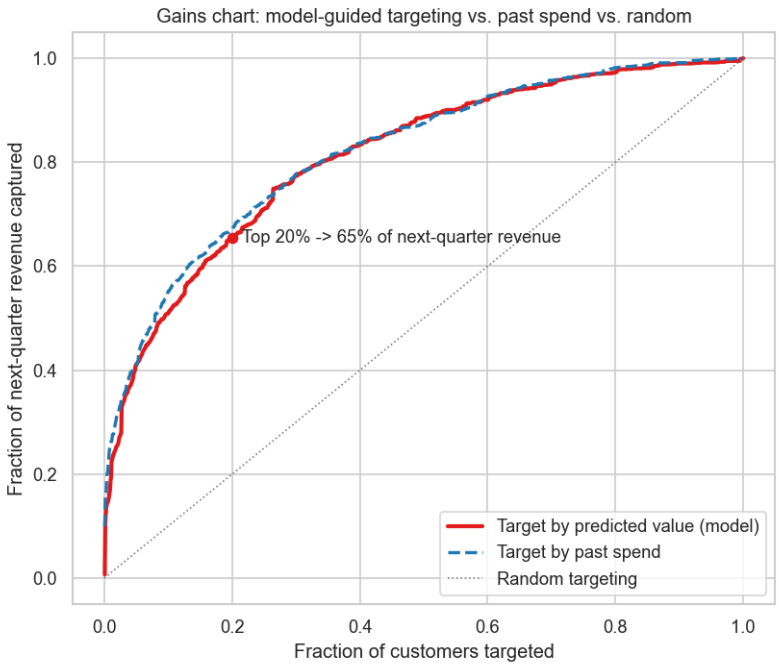

Figure 4. Gains chart. Targeting by predicted value (red) captures the majority of revenue from a small fraction of customers.

Grouping the test customers into deciles of predicted value confirms the ranking is monotonic: the top decile's mean actual next-quarter revenue is about **50 times** the bottom decile's (Figure 5).

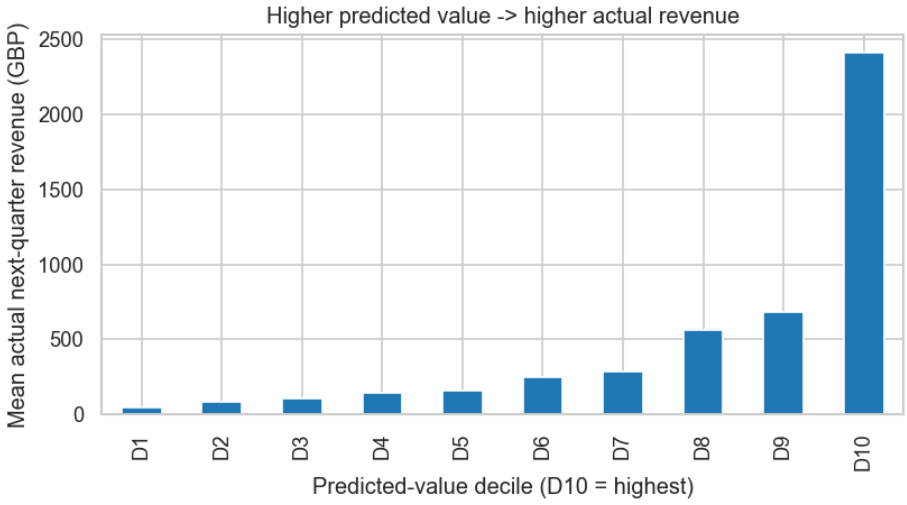

Figure 5. Mean actual next-quarter revenue rises sharply across predicted-value deciles.

5. Business impact and recommended actions

- **Concentrate spend.** Roughly two-thirds of next-quarter revenue comes from the top 20% of customers the model identifies - marketing can focus acquisition-style budgets there instead of spreading them evenly.

- **Run a targeted retention campaign.** The model flags **44 historically high-value customers** (spend above the GBP 2,118 75th-percentile) who are predicted to lapse, together worth **GBP 273,001** in the 25% hold-out alone (about GBP 1.1M across the full base). This is a named, prioritised list that a static past-spend report cannot produce.
- **Choose the simpler tool where it wins.** Because past spend ranks revenue almost as well, the model earns its place specifically through the churn probability; for pure value ranking a transparent RFM rule is a defensible, cheaper option.

6. Limitations

- A single retailer over two years; patterns may not transfer to other businesses or periods.
- The 'at-risk revenue' figure uses historic spend as a proxy for value at stake, not a formal expected-loss calculation.
- The model captures association, not causation - contacting a customer is not guaranteed to change their behaviour; an A/B test would be the proper next step.
- Predicted spend amounts are approximate (the target is noisy and zero-inflated); the ranking is more reliable than the absolute predictions.

7. Conclusion

Using SQL for feature engineering and Python for modelling, this project turns two years of raw transactions into forward-looking, actionable customer intelligence: who will return (ROC-AUC 0.79), who is most valuable (top 20% capture 65% of next-quarter revenue), and which valuable customers are slipping away. The analysis is transparent about where a simple heuristic already suffices and where the model genuinely adds value. A natural next step is an A/B test of a model-guided retention campaign to measure causal uplift.

Appendix A - SQL feature engineering (sql/build_features.sql)

```
WITH cal AS (   -- calibration window: behaviour we learn from
    SELECT * FROM transactions WHERE invoice_date < '2011-09-09'
),
pred AS (       -- prediction window: the value we want to forecast
    SELECT customer_id, SUM(revenue) AS future_revenue
    FROM transactions
    WHERE invoice_date >= '2011-09-09'
    GROUP BY customer_id
)
SELECT
    c.customer_id,
    CAST(julianday('2011-09-09') - julianday(MAX(c.invoice_date)) AS INT) AS recency_days,
    CAST(julianday('2011-09-09') - julianday(MIN(c.invoice_date)) AS INT) AS tenure_days,
    COUNT(DISTINCT c.invoice)                       AS frequency,
    ROUND(SUM(c.revenue), 2)                        AS monetary_total,
    ROUND(SUM(c.revenue) * 1.0 / COUNT(DISTINCT c.invoice), 2) AS avg_order_value,
    SUM(c.quantity)                                 AS total_items,
    COUNT(DISTINCT c.stock_code)                    AS distinct_products,
    COUNT(DISTINCT strftime('%Y-%m', c.invoice_date)) AS active_months,
    ROUND(SUM(CASE WHEN c.invoice_date >= '2011-06-11' THEN c.revenue ELSE 0 END), 2)
                                                    AS recent_revenue_90d,
    COALESCE(ROUND(p.future_revenue, 2), 0)         AS future_revenue
FROM cal c
LEFT JOIN pred p ON c.customer_id = p.customer_id
GROUP BY c.customer_id;
```

Appendix B - Data engineering script (01_build_features.py)

```
"""
Step 1 — Data engineering with SQL.
Loads the Online Retail II transactions into a SQLite database, then uses SQL to
build a customer-level feature table:
  * Features come from a CALIBRATION window (everything up to a cutoff date).
  * The TARGET is each customer's revenue in the next 91 days (PREDICTION window).
This is the standard predictive-LTV setup: learn from past behaviour, predict
future value.

Run:  python 01_build_features.py
Outputs: data/retail.db, data/customer_features.csv, sql/build_features.sql
"""
import sqlite3
from pathlib import Path
import pandas as pd

DATA = Path("data"); SQLDIR = Path("sql")
PRED_WINDOW_DAYS = 91          # predict the next ~quarter
RECENT_WINDOW_DAYS = 90        # "recent activity" window before the cutoff

con = sqlite3.connect(DATA / "retail.db")
has_table = con.execute(
    "SELECT name FROM sqlite_master WHERE type='table' AND name='transactions'"
).fetchone()

# ---------------------------------------------------------------------------
# 1. Load + clean the raw transactions (only if not already in the database)
# ---------------------------------------------------------------------------
if not has_table:
    print("Reading Excel (this takes a minute) ...")
    sheets = pd.read_excel(DATA / "online_retail_II.xlsx", sheet_name=None)
    df = pd.concat(sheets.values(), ignore_index=True)
    df.columns = [c.strip() for c in df.columns]
    df = df.rename(columns={"Customer ID": "customer_id", "Price": "price",
                            "Invoice": "invoice", "StockCode": "stock_code",
                            "Quantity": "quantity", "InvoiceDate": "invoice_date",
                            "Country": "country"})
    before = len(df)
    df = df.dropna(subset=["customer_id"])                    # need a customer
    df = df[~df["invoice"].astype(str).str.startswith("C")]  # drop cancellations
    df = df[(df["quantity"] > 0) & (df["price"] > 0)]        # drop returns/bad rows
    df["revenue"] = df["quantity"] * df["price"]
    df["customer_id"] = df["customer_id"].astype(int)
    df["invoice_date"] = pd.to_datetime(df["invoice_date"])
    print(f"Rows: {before:,} raw -> {len(df):,} clean | customers: {df.customer_id.nunique():,}")
    df[["customer_id", "invoice", "stock_code", "quantity", "price", "revenue",
        "invoice_date", "country"]].assign(
        invoice_date=lambda d: d["invoice_date"].dt.strftime("%Y-%m-%d %H:%M:%S")
    ).to_sql("transactions", con, if_exists="replace", index=False)
    con.execute("CREATE INDEX IF NOT EXISTS ix_cust ON transactions(customer_id)")
else:
    print("Reusing existing data/retail.db")

# Cutoff = last date in the data minus the prediction window
max_date = pd.to_datetime(con.execute("SELECT MAX(invoice_date) FROM transactions").fetchone()[0])
cutoff = (max_date - pd.Timedelta(days=PRED_WINDOW_DAYS)).strftime("%Y-%m-%d")
recent_start = (pd.to_datetime(cutoff) - pd.Timedelta(days=RECENT_WINDOW_DAYS)).strftime("%Y-%m-%d")
print(f"Data ends {max_date.date()} | cutoff = {cutoff} "
      f"(calibration < cutoff, target = next {PRED_WINDOW_DAYS} days)")

# ---------------------------------------------------------------------------
# 3. Feature engineering in SQL
# ---------------------------------------------------------------------------
sql = f"""
WITH cal AS (   -- calibration window: behaviour we learn from
    SELECT * FROM transactions WHERE invoice_date < '{cutoff}'
),
pred AS (       -- prediction window: the value we want to forecast
    SELECT customer_id, SUM(revenue) AS future_revenue
    FROM transactions
```

```
    WHERE invoice_date >= '{cutoff}'
    GROUP BY customer_id
)
SELECT
    c.customer_id,
    CAST(julianday('{cutoff}') - julianday(MAX(c.invoice_date)) AS INT) AS recency_days,
    CAST(julianday('{cutoff}') - julianday(MIN(c.invoice_date)) AS INT) AS tenure_days,
    COUNT(DISTINCT c.invoice)                       AS frequency,
    ROUND(SUM(c.revenue), 2)                        AS monetary_total,
    ROUND(SUM(c.revenue) * 1.0 / COUNT(DISTINCT c.invoice), 2) AS avg_order_value,
    SUM(c.quantity)                                 AS total_items,
    COUNT(DISTINCT c.stock_code)                    AS distinct_products,
    COUNT(DISTINCT strftime('%Y-%m', c.invoice_date)) AS active_months,
    ROUND(SUM(CASE WHEN c.invoice_date >= '{recent_start}' THEN c.revenue ELSE 0 END), 2)
                                                    AS recent_revenue_90d,
    COALESCE(ROUND(p.future_revenue, 2), 0)         AS future_revenue
FROM cal c
LEFT JOIN pred p ON c.customer_id = p.customer_id
GROUP BY c.customer_id;
"""
SQLDIR.mkdir(exist_ok=True)
(SQLDIR / "build_features.sql").write_text(sql.strip() + "\n")

feats = pd.read_sql(sql, con)
con.close()

feats.to_csv(DATA / "customer_features.csv", index=False)
print(f"\nCustomer feature table: {feats.shape[0]:,} customers x {feats.shape[1]} cols")
print(f"Customers active in prediction window: "
      f"{(feats.future_revenue > 0).mean():.1%}")
print("Saved data/customer_features.csv and sql/build_features.sql")
```

Appendix C - Modelling script (02_model_ltv.py)

```
"""
Step 2 — Predict customer behaviour (will they buy again?) and value (how much?),
then translate the predictions into marketing actions and measurable impact.

Two models on the same RFM-style features:
  1. CHURN / repeat-purchase classifier  -> probability a customer buys next quarter
  2. Value regressor                      -> expected next-quarter spend

Business outputs:
  * Gains chart: revenue captured by targeting the top X% of predicted-value customers.
  * At-risk list: historically high-value customers the model expects to lapse.

Run:  python 02_model_ltv.py   ->  figures + metrics.json in ./outputs
"""
import json
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import spearmanr
from sklearn.model_selection import train_test_split
from sklearn.ensemble import HistGradientBoostingRegressor, HistGradientBoostingClassifier
from sklearn.inspection import permutation_importance
from sklearn.metrics import (mean_absolute_error, roc_auc_score, average_precision_score,
                             roc_curve)

sns.set_theme(style="whitegrid")
OUT = Path("outputs"); OUT.mkdir(exist_ok=True)
RS = 42
M = {}

df = pd.read_csv("data/customer_features.csv")
FEATURES = ["recency_days", "tenure_days", "frequency", "monetary_total",
            "avg_order_value", "total_items", "distinct_products",
            "active_months", "recent_revenue_90d"]
X = df[FEATURES]
y_value = df["future_revenue"]
y_active = (df["future_revenue"] > 0).astype(int)      # behaviour target
M["n_customers"] = int(len(df))
M["active_next_quarter_pct"] = round(float(y_active.mean()), 4)

idx = np.arange(len(df))
itr, ite = train_test_split(idx, test_size=0.25, random_state=RS, stratify=y_active)

# --- skewed, zero-inflated target ----------------------------------------
plt.figure(figsize=(6, 4))
sns.histplot(np.log1p(y_value), bins=40, color="#1f78b4")
plt.xlabel("log(1 + next-quarter revenue)"); plt.ylabel("customers")
plt.title("Next-quarter revenue is skewed and zero-inflated")
plt.tight_layout(); plt.savefig(OUT / "01_target_distribution.png", dpi=120); plt.close()

# ============================================================================
# MODEL 1 — will the customer buy again next quarter? (behaviour)
# ============================================================================
clf = HistGradientBoostingClassifier(max_depth=6, learning_rate=0.06,
                                     max_iter=400, random_state=RS)
clf.fit(X.iloc[itr], y_active.iloc[itr])
p_active = clf.predict_proba(X.iloc[ite])[:, 1]
M["churn_roc_auc"] = round(float(roc_auc_score(y_active.iloc[ite], p_active)), 3)
M["churn_pr_auc"] = round(float(average_precision_score(y_active.iloc[ite], p_active)), 3)

fpr, tpr, _ = roc_curve(y_active.iloc[ite], p_active)
plt.figure(figsize=(6, 5))
plt.plot(fpr, tpr, color="#e31a1c", lw=2, label=f"Model (AUC={M['churn_roc_auc']:.2f})")
plt.plot([0, 1], [0, 1], "k--", lw=1, label="Random")
plt.xlabel("False positive rate"); plt.ylabel("True positive rate")
plt.title("Predicting who returns next quarter"); plt.legend()
```

```
plt.tight_layout(); plt.savefig(OUT / "04_churn_roc.png", dpi=120); plt.close()

# ============================================================================
# MODEL 2 — how much will they spend? (value)  [log target]
# ============================================================================
reg = HistGradientBoostingRegressor(max_depth=6, learning_rate=0.06, max_iter=500,
                                    l2_regularization=1.0, random_state=RS)
reg.fit(X.iloc[itr], np.log1p(y_value.iloc[itr]))
pred_spend = np.clip(np.expm1(reg.predict(X.iloc[ite])), 0, None)
M["value_mae"] = round(float(mean_absolute_error(y_value.iloc[ite], pred_spend)), 2)
M["value_spearman"] = round(float(spearmanr(y_value.iloc[ite], pred_spend).statistic), 3)

# Expected value = P(buy) x predicted spend  -> the ranking score for targeting
exp_value = p_active * pred_spend

# ============================================================================
# BUSINESS IMPACT 1 — gains chart (revenue captured by targeting top X%)
# ============================================================================
te = pd.DataFrame({"actual": y_value.iloc[ite].values,
                   "score": exp_value,
                   "past_spend": X.iloc[ite]["monetary_total"].values})
total = te["actual"].sum()

def gains(col):
    d = te.sort_values(col, ascending=False)
    return (np.arange(1, len(d) + 1) / len(d)), (d["actual"].cumsum().values / total)

cf, g_model = gains("score")
_, g_past = gains("past_spend")
cap = lambda frac, g: float(g[np.searchsorted(cf, frac) - 1])
M["model_capture_top20"] = round(cap(0.20, g_model), 3)
M["pastspend_capture_top20"] = round(cap(0.20, g_past), 3)
M["lift_top20_vs_random"] = round(M["model_capture_top20"] / 0.20, 2)

plt.figure(figsize=(7, 6))
plt.plot(cf, g_model, color="#e31a1c", lw=2.5, label="Target by predicted value (model)")
plt.plot(cf, g_past, color="#1f78b4", lw=2, ls="--", label="Target by past spend")
plt.plot([0, 1], [0, 1], color="grey", lw=1, ls=":", label="Random targeting")
plt.scatter([0.20], [M["model_capture_top20"]], color="#e31a1c", zorder=5)
plt.annotate(f"  Top 20% -> {M['model_capture_top20']*100:.0f}% of next-quarter revenue",
             (0.20, M["model_capture_top20"]), fontsize=11, va="center")
plt.xlabel("Fraction of customers targeted")
plt.ylabel("Fraction of next-quarter revenue captured")
plt.title("Gains chart: model-guided targeting vs. past spend vs. random")
plt.legend(loc="lower right"); plt.tight_layout()
plt.savefig(OUT / "02_gains_chart.png", dpi=120); plt.close()

# Decile lift
te["decile"] = pd.qcut(te["score"].rank(method="first"), 10,
                       labels=[f"D{i}" for i in range(1, 11)])
dec = te.groupby("decile", observed=True)["actual"].mean().reindex([f"D{i}" for i in range(1, 11)])
plt.figure(figsize=(7, 4))
dec.plot(kind="bar", color="#1f78b4")
plt.ylabel("Mean actual next-quarter revenue (GBP)")
plt.xlabel("Predicted-value decile (D10 = highest)")
plt.title("Higher predicted value -> higher actual revenue")
plt.tight_layout(); plt.savefig(OUT / "03_decile_lift.png", dpi=120); plt.close()
M["top_decile_vs_bottom_decile_x"] = round(float(dec["D10"] / max(dec["D1"], 1e-9)), 1)

# ============================================================================
# BUSINESS IMPACT 2 — high-value customers at risk of lapsing
# (something "rank by past spend" alone cannot surface)
# ============================================================================
hv_cut = float(df["monetary_total"].quantile(0.75))
test = df.iloc[ite].copy(); test["p_active"] = p_active
hv = test[test["monetary_total"] >= hv_cut]
at_risk = hv[hv["p_active"] < 0.5]
M["high_value_threshold_gbp"] = round(hv_cut, 2)
M["n_high_value_test"] = int(len(hv))
M["n_at_risk_test"] = int(len(at_risk))
M["revenue_at_risk_test_gbp"] = round(float(at_risk["monetary_total"].sum()), 2)
M["at_risk_share_of_high_value"] = round(float(len(at_risk) / max(len(hv), 1)), 3)
```

```
# Feature importance for the behaviour model (what predicts returning)
imp = permutation_importance(clf, X.iloc[ite], y_active.iloc[ite],
                             n_repeats=5, random_state=RS, n_jobs=-1)
order = np.argsort(imp.importances_mean)
plt.figure(figsize=(7, 4.2))
plt.barh(np.array(FEATURES)[order], imp.importances_mean[order], color="#1f78b4")
plt.xlabel("Permutation importance"); plt.title("What predicts a repeat purchase")
plt.tight_layout(); plt.savefig(OUT / "05_feature_importance.png", dpi=120); plt.close()

with open(OUT / "metrics.json", "w") as f:
    json.dump(M, f, indent=2)
print(json.dumps(M, indent=2))
print("\nDone. Figures + metrics.json in ./outputs")
```